

Proyecto sistemas operativos
Indicador de sentimiento económico

Omas Manuel Soto Pinzón

David Vallejo Giraldo

Nicolas Roa Hernandez

David Andrés Galindo Rojas

Sistemas Operativos

Facultad de Ingeniería

Pontificia Universidad Javeriana

Tabla de Contenido:

1. Introducción
2. Conceptos utilizados dentro del proyecto
    2.1 Procesos e Identificación
    2.2 Comunicación entre Procesos (Pipes Nominales / FIFO)
    2.3 Concurrencia con Hilos POSIX (pthread)
    2.4 Sincronización y Buffers "Productor-Consumidor"
    2.5 Gestión de Archivos y Liberación de Recursos
3. Metodología
4. Plan de pruebas
    4.1 Metodología
    4.2 Entorno de pruebas
    4.3 Generador de encuestas
    4.4 Agente de lectura
    4.5 Servidor ISE
5. Pruebas de rendimiento
    5.1 Diseño de las pruebas
    5.2 Resultados
    5.3 Efecto del tamaño del buffer
    5.4 Efecto de la cantidad de agentes
6. Análisis de resultados
    6.1 Correctitud de los componentes
    6.2 Exactitud del cálculo del ISE
    6.3 Efecto del tamaño del buffer
    6.4 Efecto de la cantidad de agentes
    6.5 Variabilidad y limitaciones de la medición
7. Conclusiones

1. Introducción

El presente proyecto desarrolla un sistema para calcular el Indicador de Sentimiento Económico (ISE) a partir de encuestas simuladas de España, Francia e Italia, correspondientes a los sectores de industria, servicios, consumidores, comercio y construcción. Su objetivo es aplicar los conceptos de comunicación entre procesos, concurrencia y sincronización, mediante una implementación en lenguaje C sobre Linux.

El sistema está compuesto por tres programas: un generador de archivos CSV, un agente de lectura y un servidor encargado del procesamiento. El generador produce las encuestas según el país y el porcentaje solicitado; cada agente lee un archivo y envía sus registros al servidor mediante un pipe nominal, el servidor recibe la información y la distribuye en buffers independientes, atendidos por hilos y sincronizados mediante semáforos POSIX y mutex.

A partir de los datos recibidos, el servidor calcula los promedios por sector y aplica sus respectivos pesos para obtener el ISE. Además, presenta reportes parciales cuando finaliza cada agente y un resumen general al concluir el tiempo de ejecución. Este documento describe el diseño, la implementación y las pruebas del sistema, explicando cómo se coordinan sus componentes para procesar las encuestas.

2. Conceptos utilizados dentro del proyecto

2.1 Procesos e Identificación

El proyecto funciona bajo un modelo Cliente-Servidor dividido en tres programas independientes que se ejecutan como procesos separados en el sistema operativo:

Generador (generadorCSV**):** Crea los archivos de datos con las encuestas de cada país.

Agentes (agentePais**):** Actúan como clientes, en este varios agentes pueden ejecutarse al mismo tiempo desde distintas terminales para leer los archivos CSV y enviar las encuestas al servidor. Cada agente se identifica ante el servidor usando su ID de proceso (PID).

Servidor (servidorISE**):** Es el proceso central que recibe la información de todos los agentes, organiza los datos por país y calcula el Indicador de Sentimiento Económico ISE.

2.2 Comunicación entre Procesos (Pipes Nominales / FIFO)

Como los agentes y el servidor son programas independientes, utilizan un Pipe Nominal (FIFO) para comunicarse, este mecanismo funciona como un canal o tubería compartida dentro del sistema de archivos donde varios agentes pueden depositar mensajes y el servidor los va leyendo en orden de llegada.

Para mantener el orden en la transmisión, la comunicación se divide en tres tipos de mensajes: un mensaje de inicio que es cuando el agente se conecta, los mensajes de encuesta, que son los datos como tal y un mensaje de fin siendo cuando el agente termina su envío.

2.3 Concurrencia con Hilos POSIX (pthread)

Para que el servidor pueda realizar varias tareas al mismo tiempo sin quedarse bloqueado esperando una sola acción, el trabajo interno se divide en varios hilos de ejecución:

Hilo de Tiempo (Reloj): Lleva la cuenta regresiva del tiempo máximo de ejecución configurado para el servidor y avisa cuando debe detenerse.

Hilo Recolector: Se encarga únicamente de escuchar el Pipe Nominal, recibir las encuestas que mandan los agentes y repartirlas según el país al que pertenecen.

Hilos de Agentes: Por cada país que se conecta, el servidor crea un hilo dedicado exclusivamente a procesar sus encuestas, calcular sus promedios por sector y generar su reporte parcial.

2.4 Sincronización y Buffers "Productor-Consumidor"

Como varios hilos trabajan al mismo tiempo dentro del servidor, se implementaron mecanismos de sincronización para evitar errores o pérdida de datos:

Buffers Circulares: Cada agente conectado tiene una fila de espera (buffer) en memoria donde el hilo recolector guarda las encuestas recién llegadas mientras el hilo del agente las va procesando.

Semáforos: Controlan el espacio del buffer; en caso de que el buffer se llena, el recolector espera antes de meter más datos; si está vacío, el hilo del agente espera hasta que llegue una encuesta nueva.

Mutex: Funciona como un candado para asegurar que dos hilos no modifiquen la misma información al mismo tiempo ni impriman reportes mezclados en la pantalla.

2.5 Gestión de Archivos y Liberación de Recursos

El sistema administra de forma ordenada tanto los archivos como la memoria que utiliza durante la ejecución:

> **Manejo de Archivos:** Se emplea lectura y escritura de archivos para crear y leer los registros en formato CSV de cada país.
>
> **Cierre Limpio:** Al terminar el tiempo de ejecución o finalizar los envíos, el servidor espera a que todos los hilos terminen su trabajo pendiente, cierra los archivos abiertos, libera la memoria reservada para los buffers y elimina la tubería de comunicación (FIFO) para no dejar recursos ocupados en el sistema.

3. Metodología

El proyecto se dividió en componentes independientes que posteriormente fueron integrados mediante mecanismos de comunicación y sincronización proporcionados por POSIX.

En una primera etapa se realizó el análisis de los requerimientos y se identificaron los tres componentes principales del sistema: el generador de encuestas, los agentes de lectura y el servidor principal ISE. Esta división permitió trabajar cada funcionalidad por separado y comprobar su funcionamiento antes de realizar la integración completa.

El primer componente desarrollado fue el generadorCSV, encargado de recibir mediante argumentos el país y el porcentaje de encuestas que se desea generar. A partir de estos valores se calcula la cantidad de registros correspondiente a cada sector económico, respetando los máximos establecidos para España, Francia e Italia. Posteriormente se generan valores aleatorios de sentimiento y se almacenan en archivos CSV con una estructura común para todos los países.

Después se desarrolló agentePais, encargado de actuar como cliente del sistema. Este programa recibe el nombre de un archivo CSV y el nombre del pipe nominal mediante argumentos de línea de comandos. El agente valida los parámetros recibidos, abre el archivo, determina la cantidad total de encuestas y convierte cada línea en una estructura de datos. Posteriormente se conecta con el servidor mediante un pipe nominal y envía la información utilizando un protocolo compuesto por mensajes de inicio, encuesta y finalización. Durante este proceso se muestra en consola el progreso del envío cada 100 registros. Para identificar cada agente durante la comunicación se utiliza el PID del proceso, obtenido mediante getpid(). Esta información se incluye en los mensajes enviados al servidor junto con los datos correspondientes al país y a las encuestas procesadas.

En la siguiente etapa se desarrolló servidorISE, encargado de crear el pipe nominal y recibir la información enviada por los diferentes agentes. El servidor utiliza hilos POSIX para separar las tareas de recepción, procesamiento y control del tiempo de ejecución. Además, se

implementan buffers independientes para almacenar temporalmente las encuestas correspondientes a cada agente.
Debido a que varios hilos pueden acceder simultáneamente a recursos compartidos, se emplean semáforos POSIX para controlar el acceso a los buffers y evitar condiciones de carrera. De esta forma se garantiza que la lectura y escritura de información se realicen de manera segura durante la ejecución concurrente del sistema.

Una vez recibidas las encuestas, el servidor procesa la información de acuerdo con los sectores económicos establecidos en el proyecto: industria, servicios, consumidores, comercio y construcción. Cada sector tiene un peso diferente dentro del cálculo general del indicador, correspondiente a 40 %, 30 %, 20 %, 5 % y 5 %, respectivamente. Con estos datos se generan reportes parciales por agente y posteriormente un resultado general del ISE.

También se implementó un control de tiempo para limitar la ejecución del servidor al intervalo indicado mediante los parámetros de entrada. La lectura de información y el control del tiempo se realizan mediante hilos POSIX diferentes, tal como lo establece el enunciado.
Durante todas las etapas se realizaron pruebas de manera incremental. Inicialmente se verificó cada programa de manera independiente y posteriormente se realizaron pruebas de integración utilizando uno y varios agentes. Se comprobaron casos como argumentos incorrectos, archivos inexistentes, porcentajes fuera de rango, comunicación mediante el pipe, envío completo de encuestas, finalización de agentes y liberación de recursos.

Finalmente, se utilizó un Makefile para automatizar la compilación de los tres programas principales y mantener una configuración uniforme.

Esta metodología permitió construir el sistema de forma progresiva, reduciendo la dificultad de depuración y facilitando la integración de los componentes de comunicación, concurrencia y sincronización requeridos.

4. Plan de pruebas

4.1 Metodología

Un plan de pruebas es el conjunto de verificaciones que se realizan a las operaciones principales del programa para garantizar su correcto funcionamiento ante diferentes valores de entrada. Para cada operación se define un plan con al menos tres casos de prueba.

En cada caso se describe la situación a probar, se indican los valores de entrada, se calcula el resultado esperado y se registra el resultado obtenido al ejecutar el programa. Como evidencia se adjunta una captura de pantalla de la ejecución de cada caso.

Las pruebas cubren los tres componentes del sistema:

-Generador de encuestas: planes G1 y G2.

-Agente de lectura: planes A1 y A2.
-Servidor Control Proceso Principal ISE: planes S1 a S4.

Adicionalmente, la sección 5 presenta pruebas de rendimiento que miden cómo responde el sistema al variar el tamaño del buffer y la cantidad de agentes.

4.2 Entorno de pruebas

Elemento	Valor
Sistema operativo	Ubuntu 26.04.1 LTS
Versión del kernel	6.18.33.2-microsoft-standard-WSL2
Tipo de máquina	Virtual: WSL 2 (versión 2.7.11.0) sobre Windows
Procesador	Intel Core Ultra 7 265KF (arquitectura híbrida: núcleos de rendimiento y de eficiencia)
Núcleos asignados	20 (20 núcleos físicos, 1 hilo por núcleo)
Memoria RAM	31 GiB asignados a WSL
Compilador	gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0

4.3 Generador de encuestas

G1. Validación de argumentos del generador

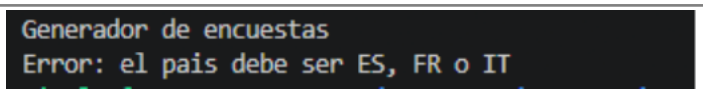
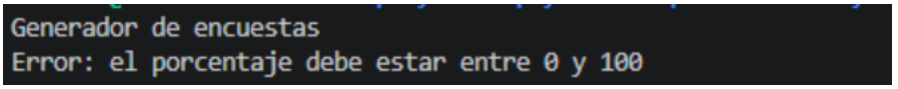
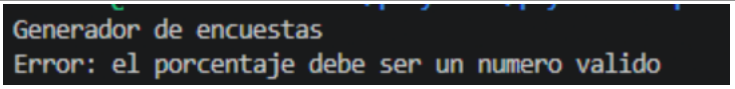
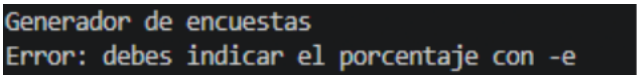
Qué mide: que el generador rechace entradas inválidas con un mensaje claro y sin crear archivos.

Plan de pruebas: Validación de argumentos del generador

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
País inválido	-p CO -e 10	Error: el país debe ser ES, FR o IT	Error: el país debe ser ES, FR o IT
Porcentaje negativo	-p ES -e -5	Error: el porcentaje debe estar entre 0 y 100	Error: el porcentaje debe estar entre 0 y 100
Porcentaje no numérico	-p ES -e abc	Error: el porcentaje debe ser un número válido	Error: el porcentaje debe ser un número válido
Falta una bandera	-p ES	Error: debes indicar el porcentaje con -e	Error: debes indicar el porcentaje con -e

Evidencias: Validación de argumentos del generador

Caso	Evidencia de ejecución

País inválido	
Porcentaje negativo	
Porcentaje no numérico	
Falta una bandera	

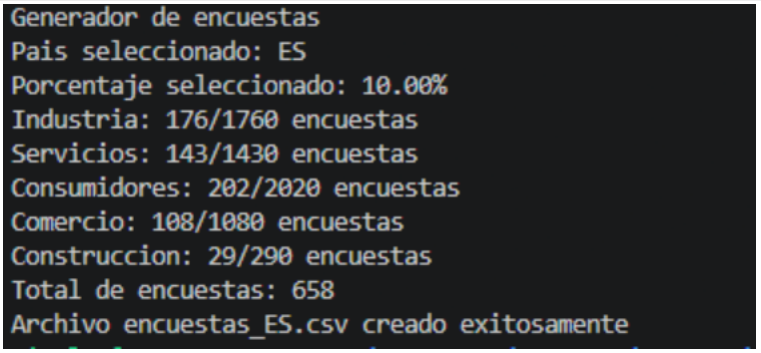
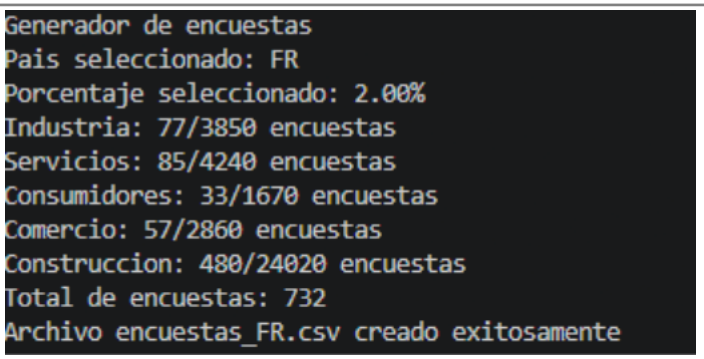
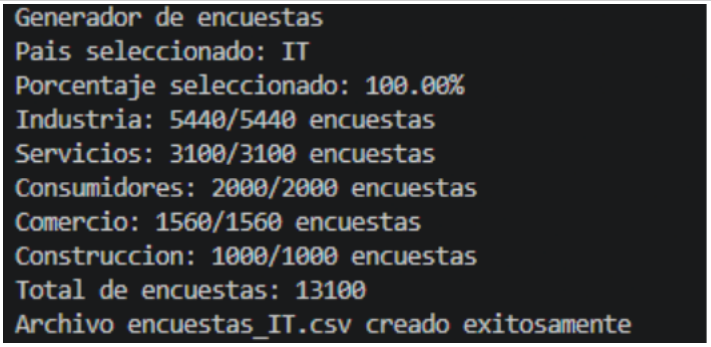
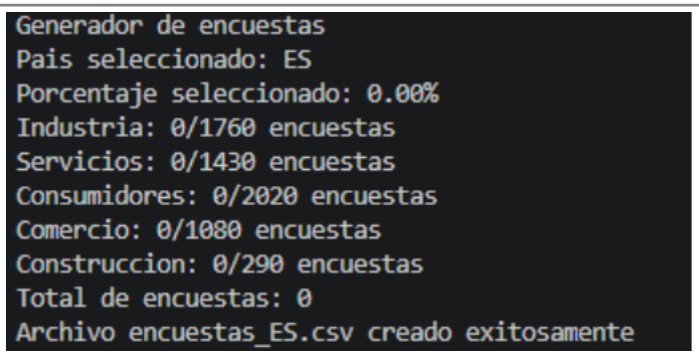
G2. Cantidad de encuestas por sector

Qué mide: que la cantidad de encuestas por sector sea el porcentaje indicado de los máximos de la Tabla 1 del enunciado, redondeado.

Plan de pruebas: Cantidad de encuestas por sector

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
Porcentaje normal	-p ES -e 10	176 / 143 / 202 / 108 / 29 = 658 encuestas	176 / 143 / 202 / 108 / 29 = 658 encuestas
Porcentaje con redondeo	-p FR -e 2	77 / 85 / 33 / 57 / 480 = 732 encuestas	77 / 85 / 33 / 57 / 480 = 732 encuestas
Porcentaje máximo	-p IT -e 100	5440 / 3100 / 2000 / 1560 / 1000 = 13100 encuestas	5440 / 3100 / 2000 / 1560 / 1000 = 13100 encuestas
Porcentaje nulo	-p ES -e 0	0 en todos los sectores; archivo solo con la cabecera	0 en todos los sectores; archivo solo con la cabecera

Evidencias: Cantidad de encuestas por sector

Caso	Evidencia de ejecución
Porcentaje normal	
Porcentaje con redondeo	
Porcentaje máximo	
Porcentaje nulo	

4.4 Agente de lectura

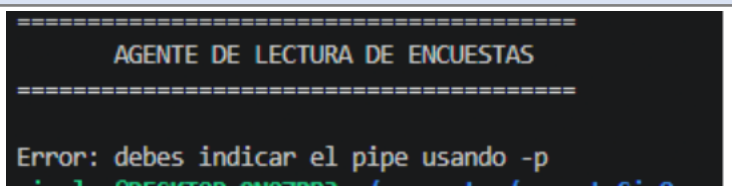
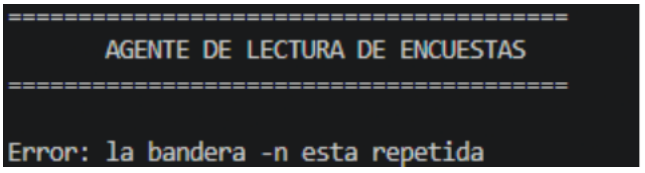
A1. Validación de argumentos y archivo del agente

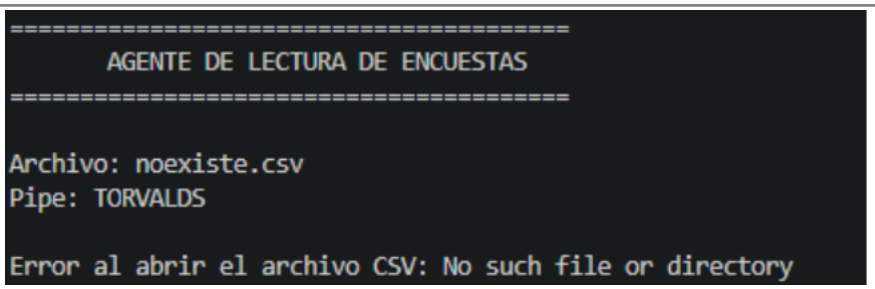
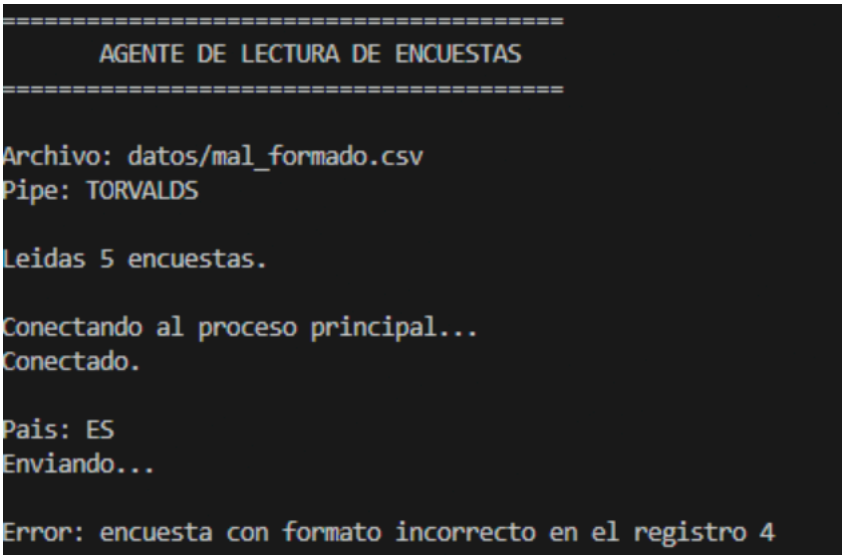
Qué mide: que el agente rechace argumentos incompletos o repetidos y detecte archivos inexistentes o mal formados.

Plan de pruebas: Validación de argumentos y archivo del agente

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
Falta el pipe	-n encuestas_ES.csv	Error: debes indicar el pipe usando -p	Error: debes indicar el pipe usando -p
Bandera repetida	-n a.csv -n b.csv -p TORVALDS	Error: la bandera -n está repetida	Error: la bandera -n está repetida
Archivo inexistente	-n noexiste.csv -p TORVALDS	Error al abrir el archivo CSV: No such file or directory	Error al abrir el archivo CSV: No such file or directory
Línea mal formada	datos/mal_formado.csv (5 encuestas, la 4.ª con 4 campos)	Agente: error de formato en el registro 4. Servidor: se recibieron 3 de 5 anunciadas -> SIN FIN	Agente: error de formato en el registro 4. Servidor: se recibieron 3 de 5 anunciadas -> SIN FIN

Evidencias: Validación de argumentos y archivo del agente

Caso	Evidencia de ejecución
Falta el pipe	
Bandera repetida	

Archivo inexistente	
Línea mal formada	 

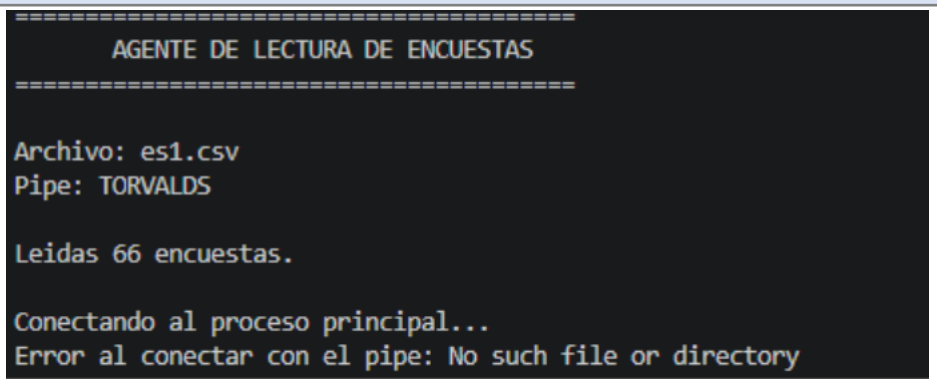
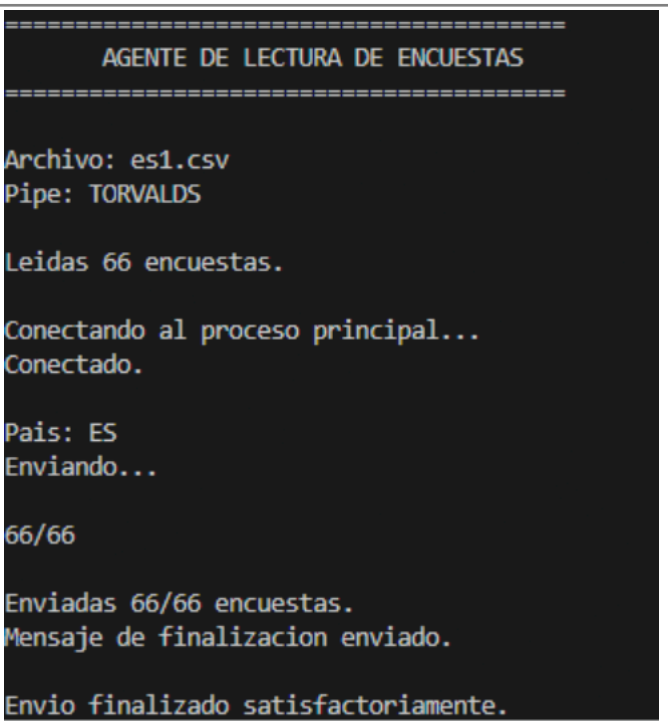
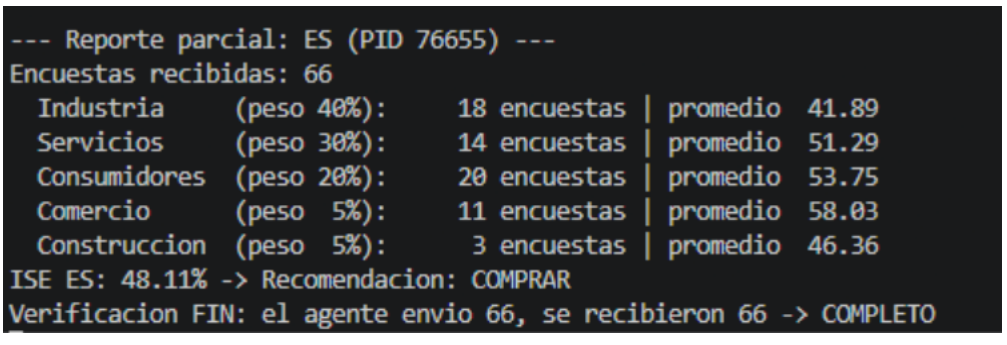
A2. Conexión y envío del agente

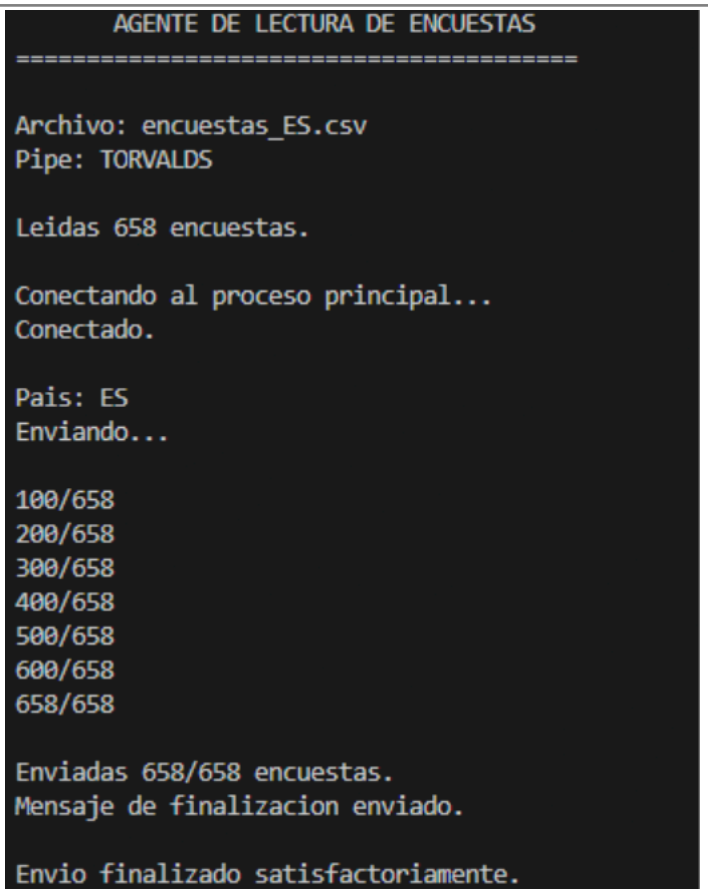
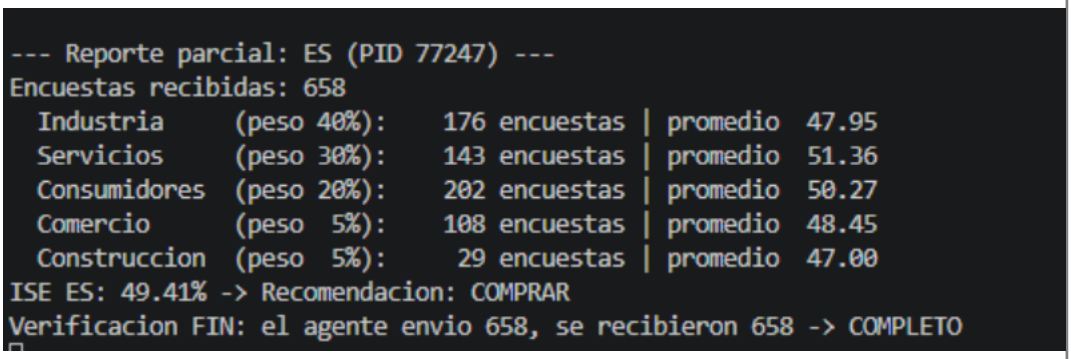
Qué mide: que el agente detecte la ausencia del servidor y reporte el progreso del envío por lotes de 100 encuestas.

Plan de pruebas: Conexión y envío del agente

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
Servidor no iniciado	-n es1.csv -p TORVALDS, sin servidor	Error al conectar con el pipe: No such file or directory	Error al conectar con el pipe: No such file or directory
Archivo pequeño	es1.csv (ES al 1 %, 66 encuestas)	Progreso solo al final: 66/66; Enviadas 66/66	Progreso solo al final: 66/66; Enviadas 66/66
Archivo con varios lotes	encuestas_ES.csv (658 encuestas)	Progreso 100/658, 200/658, …, 600/658 y 658/658	Progreso 100/658, 200/658, …, 600/658 y 658/658

Evidencias: Conexión y envío del agente

Caso	Evidencia de ejecución
Servidor no iniciado	
Archivo pequeño	 

Archivo con varios lotes	 

4.5 Servidor ISE

S1. Validación de argumentos del servidor

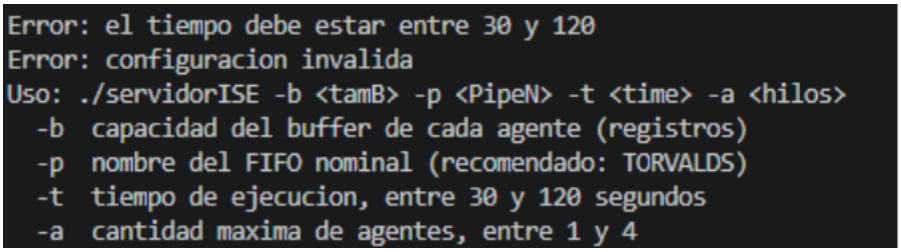
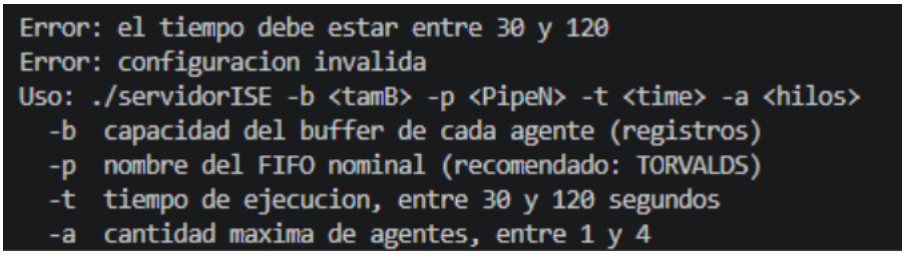
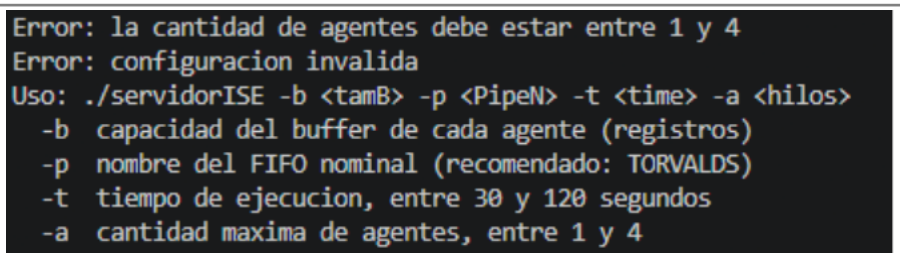
Qué mide: que el servidor acepte las banderas en cualquier orden y rechace valores fuera de los rangos del enunciado (tiempo de 30 a 120 s y máximo 4 agentes).

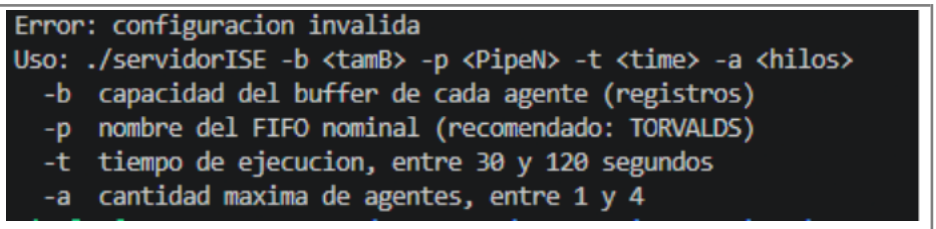
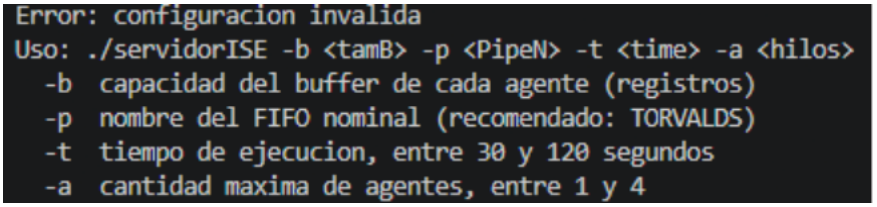
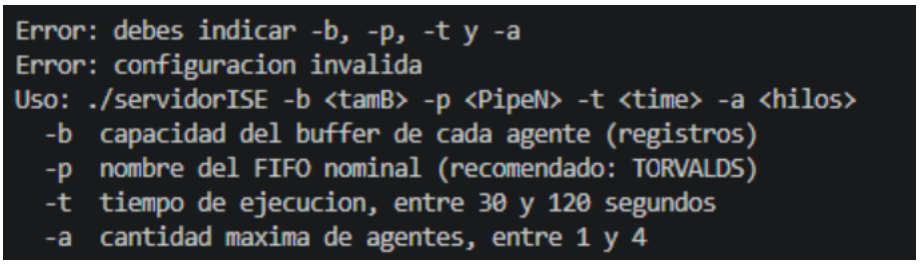
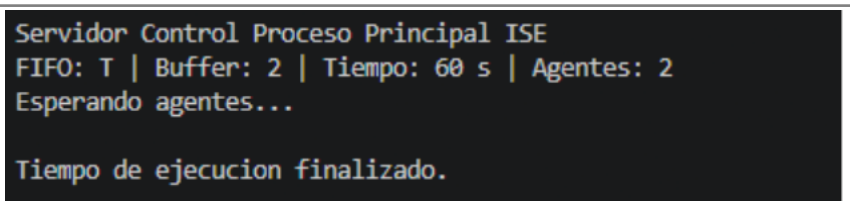
Plan de pruebas: Validación de argumentos del servidor

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
Tiempo bajo el mínimo	-b 2 -p T -t 29 -a 2	Error: el tiempo debe estar entre 30 y 120	Error: el tiempo debe estar entre 30 y 120

Tiempo sobre el máximo	-b 2 -p T -t 121 -a 2	Error: el tiempo debe estar entre 30 y 120	Error: el tiempo debe estar entre 30 y 120
Agentes fuera de rango	-a 0	Error: la cantidad de agentes debe estar entre 1 y 4	Error: la cantidad de agentes debe estar entre 1 y 4
Buffer invalido	-b 0	Error: configuración inválida y uso del programa	Error: configuración inválida y uso del programa
Bandera repetida	-b 2 -p T -t 60 -a 2 -b 3	Error: configuración inválida y uso del programa	Error: configuración inválida y uso del programa
Bandera faltante	-b 2 -t 60 -a 2	Error: debes indicar -b, -p, -t y -a	Error: debes indicar -b, -p, -t y -a
Banderas en otro orden	-a 2 -t 60 -p T -b 2	Arranca: Esperando agentes…	Arranca: Esperando agentes…

Evidencias: Validación de argumentos del servidor

Caso	Evidencia de ejecución
Tiempo bajo el mínimo	
Tiempo sobre el máximo	
Agentes fuera de rango	

Buffer invalido	
Bandera repetida	
Bandera faltante	
Banderas en otro orden	

S2. Manejo del pipe nominal

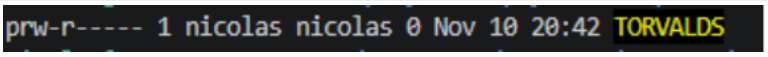
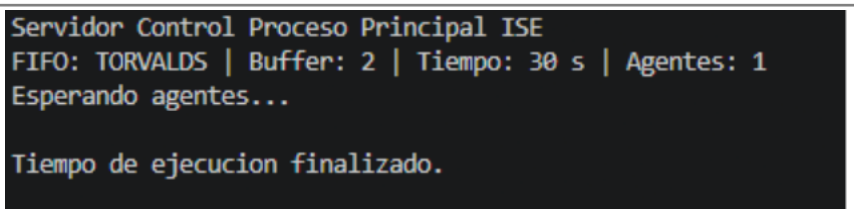
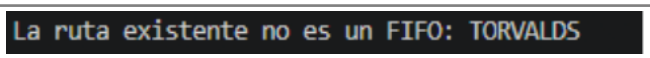
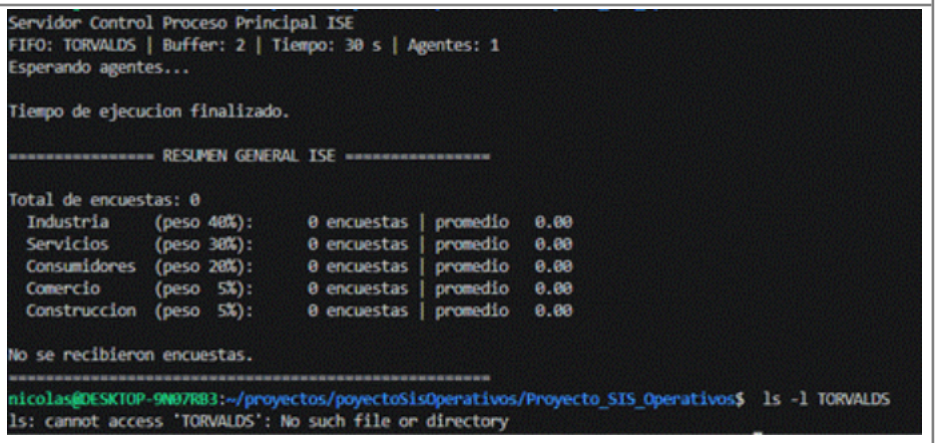
Qué mide: que el servidor cree el FIFO, reutilice uno existente, no acepte un archivo normal con ese nombre y lo borre al terminar.

Plan de pruebas: Manejo del pipe nominal

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
FIFO no existe	servidor con -p TORVALDS	Crea el FIFO: ls -l muestra una línea que empieza con p	Crea el FIFO: ls -l muestra una línea que empieza con p
FIFO ya existe	mkfifo TORVALDS; servidor con -p TORVALDS	Arranca sin error (reutiliza el FIFO)	Arranca sin error (reutiliza el FIFO)
Archivo normal con el mismo nombre	touch TORVALDS; servidor con -p TORVALDS	La ruta existente no es un FIFO: TORVALDS; no arranca	La ruta existente no es un FIFO: TORVALDS; no arranca

Fin de la ejecución	ls -l TORVALDS al terminar el servidor	No such file or directory (el FIFO fue borrado)	No such file or directory (el FIFO fue borrado)

Evidencias: Manejo del pipe nominal

Caso	Evidencia de ejecución
FIFO no existe	
FIFO ya existe	
Archivo normal con el mismo nombre	
Fin de la ejecución	

S3. Conexión de agentes y limite

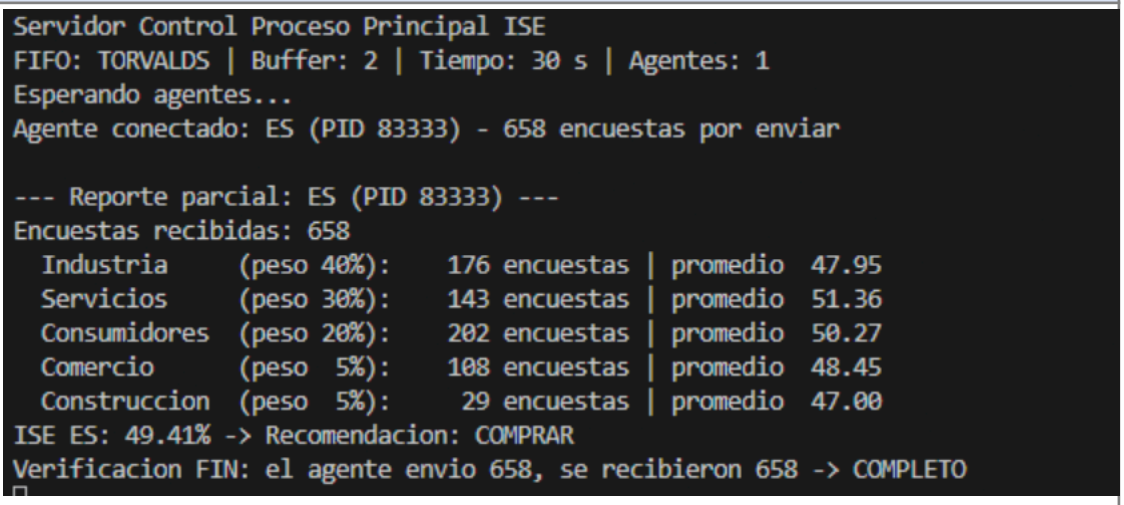
Qué mide, que el servidor registre cada agente que se conecta y rechace a los que superan el máximo indicado con -a.

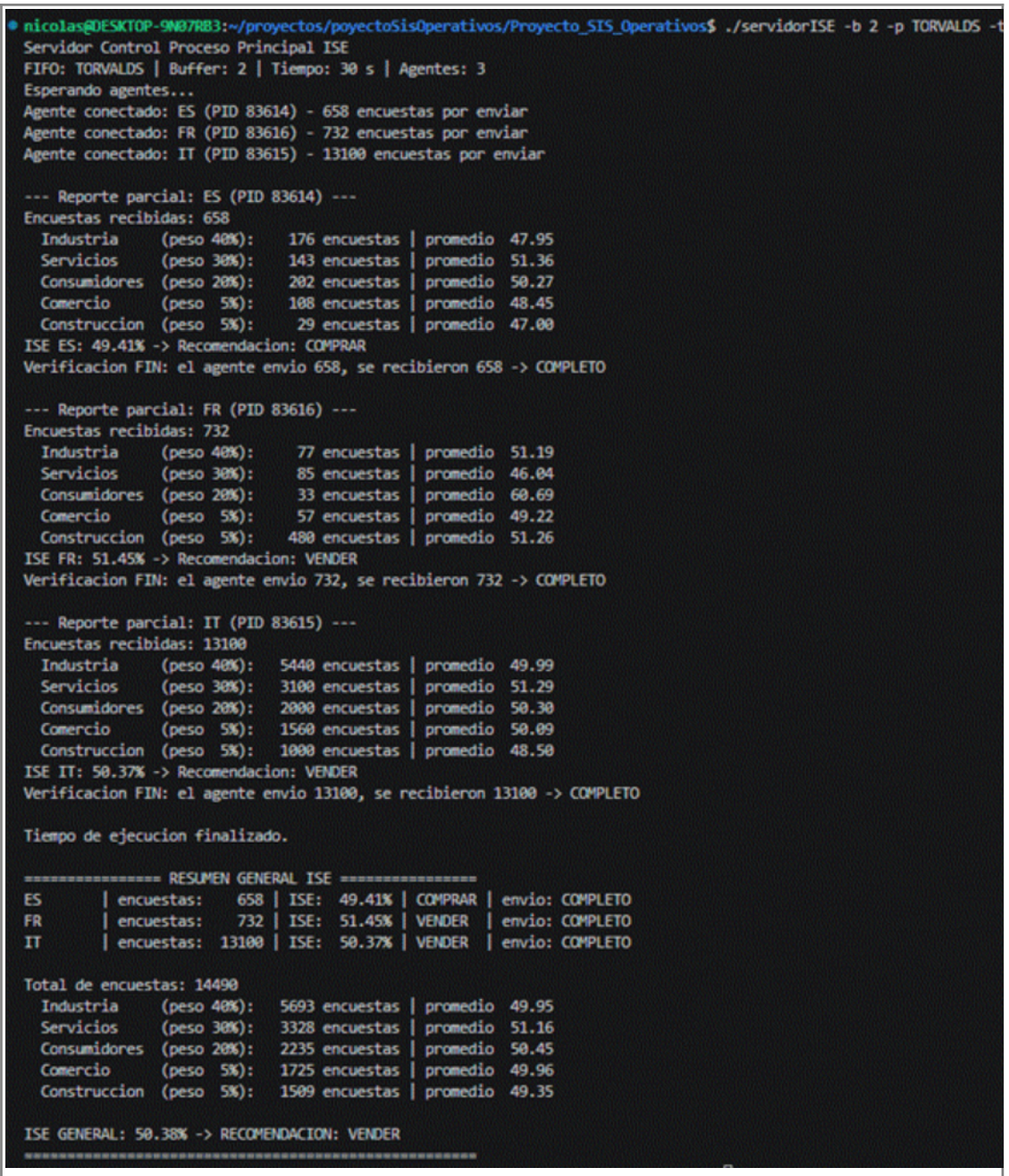
Plan de pruebas: Conexión de agentes y límite

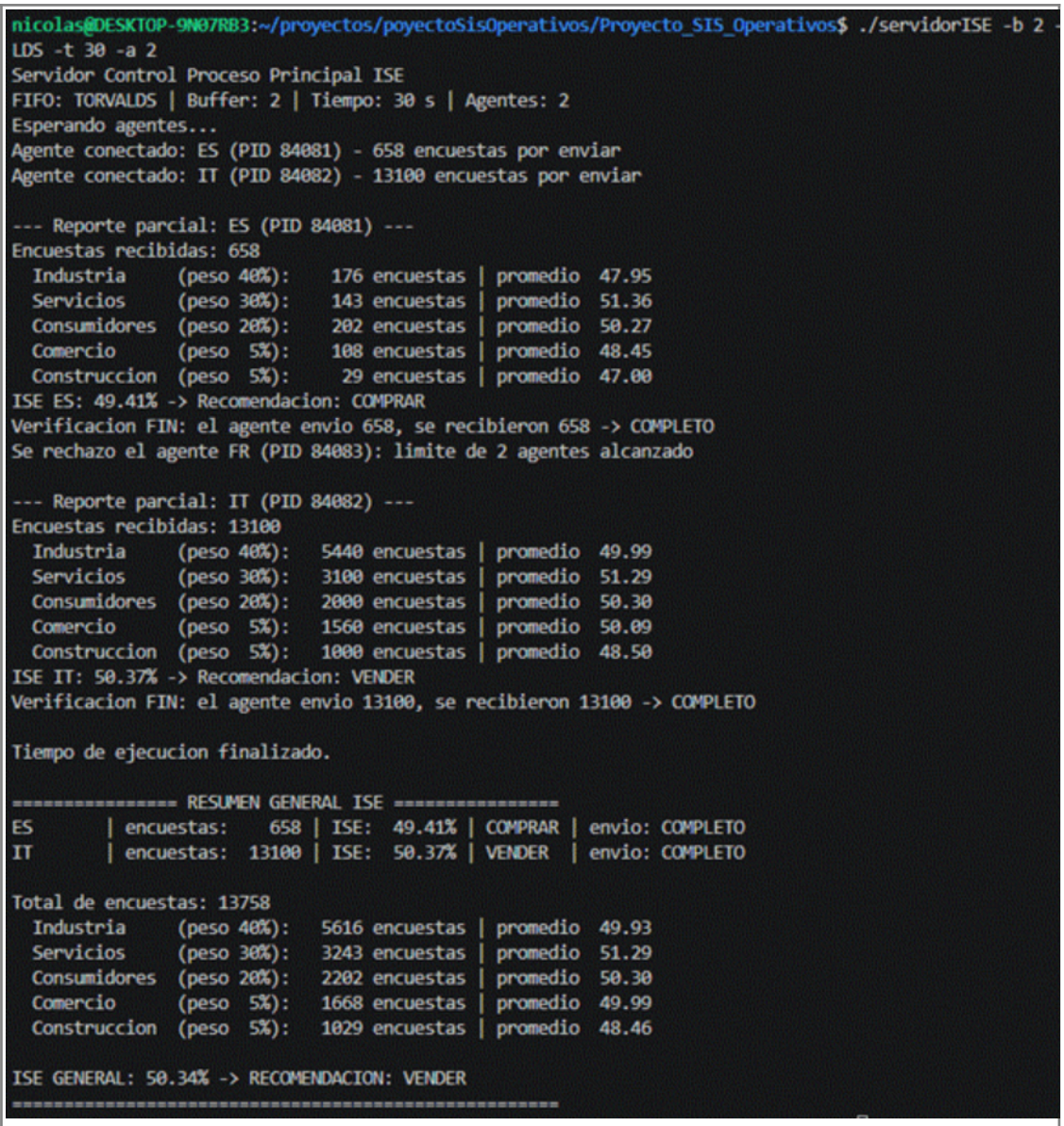
Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido

Un agente	-a 1; agente ES (658)	Agente conectado: ES (PID …) - 658 encuestas por enviar	Agente conectado: ES (PID …) - 658 encuestas por enviar
Agentes hasta el máximo	-a 3; agentes ES, IT y FR a la vez	3 mensajes Agente conectado	3 mensajes Agente conectado
Agentes por encima del máximo	-a 2; agentes ES, IT y FR a la vez	2 conectados y un solo aviso: Se rechazó el agente … límite de 2 agentes	2 conectados y un solo aviso: Se rechazó el agente … límite de 2 agentes

Evidencias: Conexión de agentes y límite

Caso	Evidencia de ejecución
Un agente	

Agentes hasta el máximo	

Agentes por encima del máximo	

S4. Cálculo del ISE y recomendación

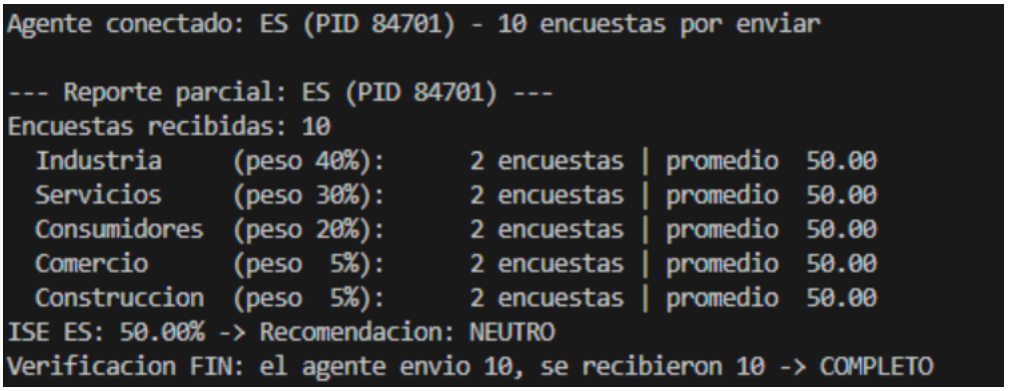
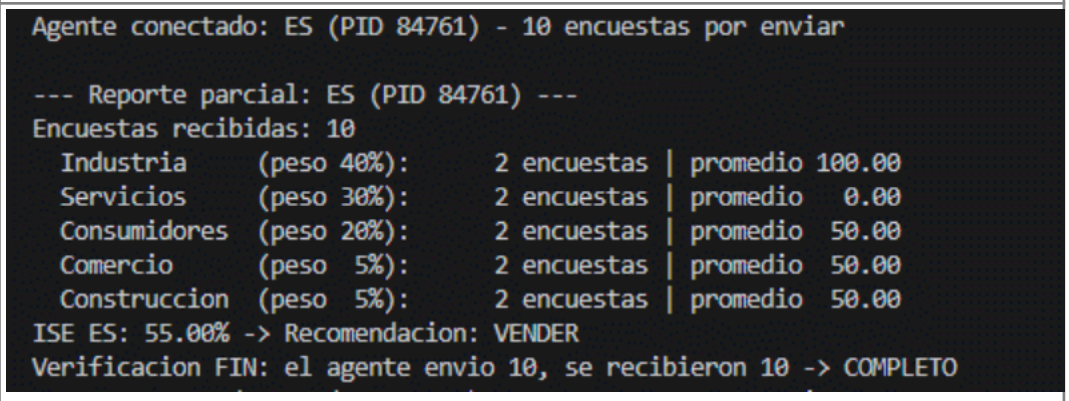
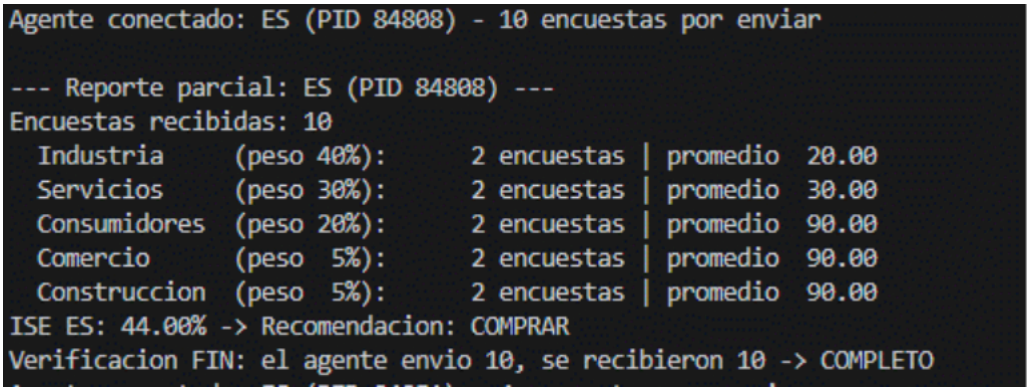
Qué mide: que el ISE sea la media ponderada de los promedios por sector (40/30/20/5/5) y que la recomendación siga el enunciado: mayor a 50 vender, menor a 50 comprar, igual a 50 neutro.

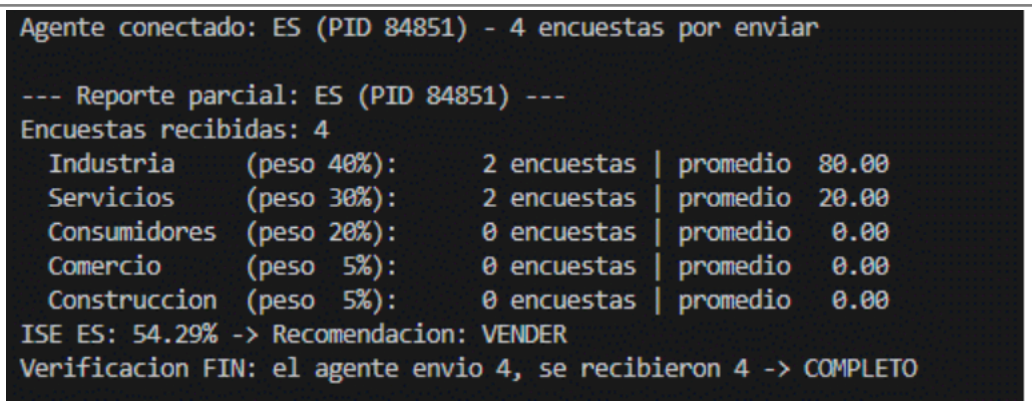
Plan de pruebas: Cálculo del ISE y recomendación

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
Sentimiento neutro	datos/ise_50.csv: todos los sectores en 50	ISE 50.00 % -> NEUTRO	ISE ES: 50.00% -> Recomendacion: NEUTRO

Ponderación favorable	datos/ise_55.csv: 100 / 0 / 50 / 50 / 50	0.4(100)+0.3(0)+0.2(50)+0.05(50)+0.05(50) = 55.00 % -> VENDER	ISE ES: 55.00% -> Recomendacion: VENDER
Ponderación desfavorable	datos/ise_44.csv: 20 / 30 / 90 / 90 / 90	8 + 9 + 18 + 4.5 + 4.5 = 44.00 % -> COMPRAR	ISE ES: 44.00% -> Recomendacion: COMPRAR
Sectores sin encuestas	datos/ise_vacios.csv: Industria 70 y 90, Servicios 10 y 30	Promedios 80 y 20; (0.4·80 + 0.3·20) / 0.7 = 54.29 % -> VENDER	Promedios 80.00 y 20.00; ISE ES: 54.29% -> Recomendacion: VENDER

Evidencias: Cálculo del ISE y recomendación

Caso	Evidencia de ejecución
Sentimiento neutro	
Ponderación favorable	
Ponderación desfavorable	

Sectores sin encuestas	

5. Pruebas de rendimiento

5.1 Diseño de las pruebas

Estas pruebas miden cuánto tarda el sistema en procesar la misma carga de encuestas al variar el tamaño del buffer de cada agente y la cantidad de agentes que envían a la vez. Se ejecutaron automáticamente con el script lanzadorISE.pl, construido con la misma estructura del lanzador del taller de evaluación de rendimiento.

1. Configuraciones: tamB con 1, 10 y 100 y agentes con 1, 2 y 4, es decir, 9 combinaciones.
1. Carga: archivo de Francia al 100 % (36 640 encuestas por agente).
1. Repeticiones: 30 por configuración (270 mediciones en total).
1. Medición: tiempo en segundos, desde que se lanzan los agentes hasta que el último termina de enviar.

5.2 Resultados

La tabla 5.1 resume los tiempos medidos en las 30 repeticiones de cada configuración: promedio, desviación estándar, mínimo y máximo.

Tabla 5.1. Tiempo de procesamiento por configuración (30 repeticiones)

tamB	Agentes	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1	1	0.7184	0.0246	0.6881	0.7896
1	2	1.4487	0.0570	1.3837	1.6764
1	4	2.9246	0.0892	2.8253	3.1600
10	1	0.0929	0.0153	0.0751	0.1217
10	2	0.1698	0.0190	0.1483	0.2151
10	4	0.3566	0.0295	0.3061	0.4244
100	1	0.0288	0.0006	0.0272	0.0299
100	2	0.0567	0.0005	0.0556	0.0576

100	4	0.1122	0.0006	0.1109	0.1134

La tabla 5.2 presenta medidas derivadas: la velocidad del servidor (encuestas procesadas por segundo, contando todos los agentes), el costo promedio de cada encuesta, cuántas veces crece el tiempo respecto a la misma configuración con 1 agente, cuántas veces es más rápido que con tamB = 1, y el coeficiente de variación (desviación / promedio).

Tabla 5.2. Medidas derivadas

tamB	Agentes	Encuestas / s	µs por encuesta	Tiempo relativo a 1 agente	Veces más rápido que tamB = 1	Coef. de variación
1	1	51 004	19.61	1.00x	—	3.4 %
1	2	50 582	19.77	2.02x	—	3.9 %
1	4	50 113	19.95	4.07x	—	3.0 %
10	1	394 572	2.53	1.00x	7.7x	16.4 %
10	2	431 490	2.32	1.83x	8.5x	11.2 %
10	4	411 016	2.43	3.84x	8.2x	8.3 %
100	1	1 270 751	0.79	1.00x	24.9x	2.1 %
100	2	1 292 264	0.77	1.97x	25.5x	0.9 %
100	4	1 306 122	0.77	3.89x	26.1x	0.5 %

5.3 Efecto del tamaño del buffer

Con un agente, el tiempo baja de 0.718 s con tamB = 1 a 0.093 s con tamB = 10 y a 0.029 s con tamB = 100. La figura 5.1 usa escala logarítmica en ambos ejes porque los tiempos cambian en más de un orden de magnitud.

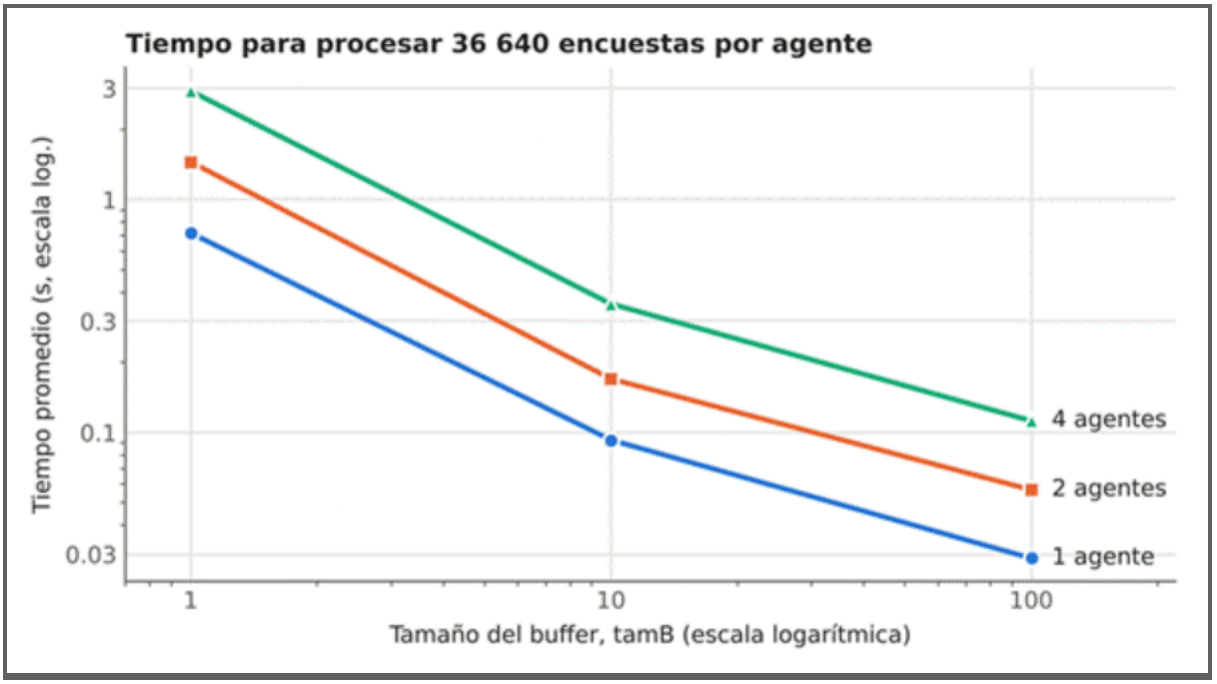


Figura 5.1. Tiempo promedio de procesamiento contra el tamaño del buffer

5.4 Efecto de la cantidad de agentes

La figura 5.2 muestra, para cada tamaño de buffer, cuántas veces crece el tiempo al pasar de 1 agente a 2 y a 4. La línea punteada es el crecimiento proporcional.

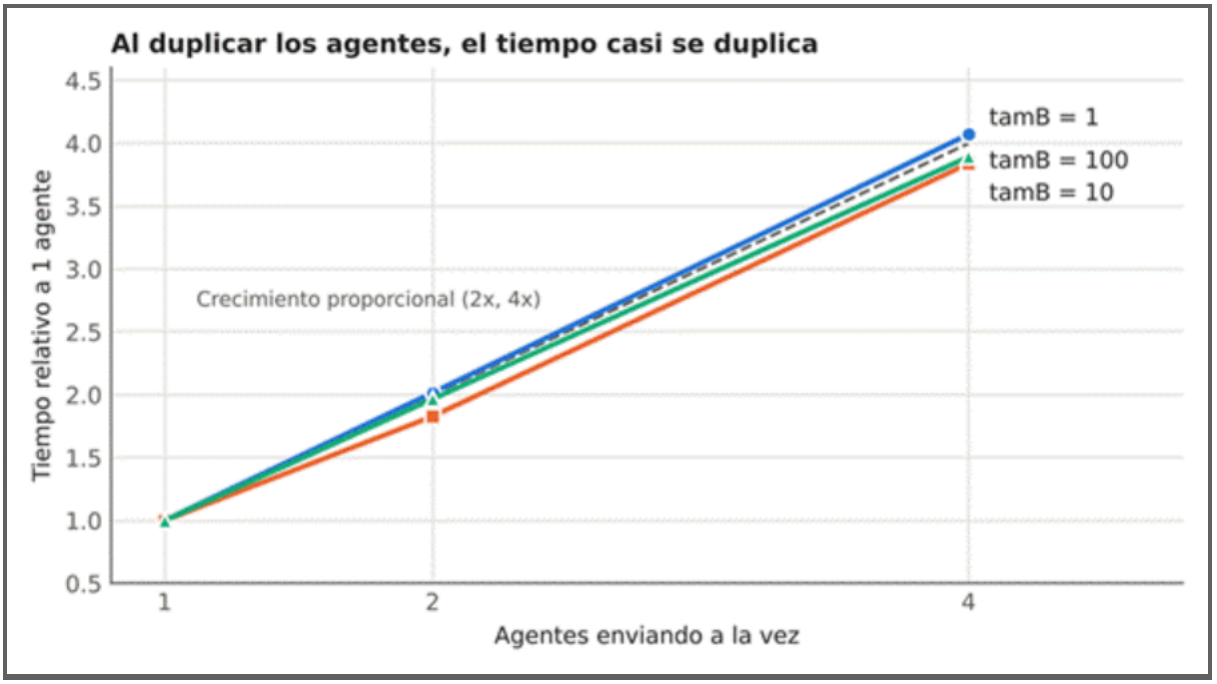


Figura 5.2. Tiempo relativo a 1 agente contra la cantidad de agentes

6. Análisis de resultados

6.1 Correctitud de los componentes

Se ejecutaron 33 casos de prueba funcionales, organizados en 8 planes, y todos dieron el resultado esperado:

Componente	Planes	Casos ejecutados	Casos correctos
Generador de encuestas	G1 y G2	8	8
Agente de lectura	A1 y A2	7	7
Servidor ISE	S1 a S4	18	18
Total	8	33	33

Los tres programas validan sus entradas, con esto rechazan los valores inválidos, nulos o fuera de rango con un mensaje claro y respetan los límites del enunciado. El generador produce exactamente las cantidades de la Tabla 1, el agente detecta archivos inexistentes, líneas mal formadas y la ausencia del servidor, y el servidor crea, reutiliza y borra el pipe nominal, no arranca si existe un archivo normal con ese nombre y rechaza con un solo aviso a los agentes que superan el límite.

6.2 Exactitud del cálculo del ISE

En los cuatro casos del plan S4, el ISE del servidor coincidió con el calculado a mano (50.00, 55.00, 44.00 y 54.29 %) y la recomendación fue la esperada. Los casos cubren las tres recomendaciones y dos situaciones delicadas:

> **El 50 % exacto:** Se compara el ISE redondeado a dos decimales, el mismo valor que se imprime, para que un 50.00 % dé NEUTRO y no VENDER.
>
> **Sectores sin encuestas:** Su peso se reparte entre los demás: con solo Industria (promedio 80) y Servicios (promedio 20), el ISE fue 54.29 % y no 38 %, que sería el resultado de contar los sectores vacíos como 0.

6.3 Efecto del tamaño del buffer

El tamaño del buffer es el factor que más influye en el rendimiento. Con un agente, pasar de tamB = 1 a tamB = 10 hace el procesamiento 7.7 veces más rápido, y pasar de 10 a 100 lo acelera 3.2 veces más en total, tamB = 100 es unas 25 veces más rápido que tamB = 1. La proporción se mantiene con 2 y 4 agentes (tabla 5.2).
La causa está en cómo se sincronizan el hilo recolector y el hilo de cada agente. Con tamB = 1, después de depositar cada encuesta el recolector queda bloqueado en sem_wait(libres) hasta que el hilo del agente la saca, y el hilo del agente queda bloqueado en

sem_wait(ocupados) hasta que llega la siguiente, cada encuesta obliga a los dos hilos a dormirse y despertarse por turnos, lo que implica al menos dos cambios de contexto por encuesta. El costo medido, 19.6 µs por encuesta, es coherente con el costo de los cambios de contexto visto en clase (entre 1 y 1000 µs).

Con un buffer más grande, el recolector puede depositar muchas encuestas seguidas sin bloquearse, y el hilo del agente las consume en ráfaga cuando le toca el procesador. Los bloqueos y cambios de contexto se reparten entre muchas encuestas, y el costo baja a 2.53 µs con tamB = 10 y a 0.79 µs con tamB = 100.

La mejora es decreciente, multiplicar el buffer por 10 aceleró unas 8 veces la primera vez, pero solo unas 3 veces la segunda. Con buffers grandes, el costo de sincronización deja de dominar y lo que queda es el costo fijo de leer cada mensaje del pipe (una llamada read por encuesta) y de acumularlo. Por eso no se espera una mejora apreciable con buffers mucho mayores que 100.

En la práctica, el costo en memoria de un buffer grande es mínimo, donde 100 posiciones de 72 bytes ocupan unos 7 KB por agente. Para este sistema conviene usar un tamB del orden de 100; un buffer de 1 o 2 posiciones hace que el servidor pase la mayor parte del tiempo sincronizando hilos en lugar de procesar encuestas.

6.4 Efecto de la cantidad de agentes

Al aumentar los agentes, el tiempo crece casi en proporción: con 2 agentes tarda entre 1.83 y 2.02 veces lo que tarda con uno, y con 4 agentes entre 3.84 y 4.07 veces (figura 5.2). Dicho de otra forma, la velocidad del servidor se mantiene prácticamente constante para cada tamaño de buffer, sin importar cuántos agentes envíen: cerca de 51 004 encuestas por segundo con tamB = 1, unas 431 490 con tamB = 10 y unas 1 306 122 con tamB = 100.

Esto muestra que el procesamiento está limitado por una parte secuencial del diseño; todos los agentes escriben en un solo pipe y un solo hilo recolector lee los mensajes uno por uno y los reparte. Aunque cada agente tiene su propio hilo en el servidor, el trabajo de esos hilos es mínimo frente al de leer y repartir, los agentes adicionales no agregan capacidad, sino que se reparten la misma capacidad del recolector.

La falta de procesadores no explica este resultado: la máquina de pruebas tiene 20 núcleos, más que suficientes para que los 4 agentes, el hilo recolector y los 4 hilos de agente del servidor corran al mismo tiempo en núcleos distintos, aun así la velocidad no aumentó con más agentes, lo que confirma que el límite está en el diseño (un solo canal y un solo lector) y no en el hardware.

El tamaño del buffer compensa en buena medida la cantidad de agentes, cuatro agentes con tamB = 10 (0.357 s) terminan en la mitad del tiempo que un solo agente con tamB = 1 (0.718 s), y cuatro agentes con tamB = 100 (0.112 s) tardan prácticamente lo mismo que un agente con tamB = 10 (0.093 s).

Si el sistema necesitara atender muchos más agentes, la mejora no vendría de agregar hilos de agente, sino de reducir el trabajo secuencial del recolector., como por ejemplo, leer varios mensajes en cada llamada read, o usar más de un pipe con su propio hilo recolectorl, con el

máximo de 4 agentes que fija el enunciado, el diseño actual procesa la carga completa (146 560 encuestas) en menos de 0.12 s con tamB = 100.

6.5 Variabilidad y limitaciones de la medición

Las mediciones son muy consistentes: con tamB = 1 y tamB = 100 el coeficiente de variación está entre 0.5 % y 4 %. La excepción es tamB = 10, con un coeficiente de variación entre 8.3 % y 16.4 %. Una explicación probable es que, con un buffer intermedio, que el recolector llegue a bloquearse depende de si el planificador del sistema operativo le da el procesador al hilo del agente a tiempo para vaciar el buffer, esa cantidad de bloqueos cambia de una ejecución a otra. Con tamB = 1 el recolector se bloquea siempre y con tamB = 100 casi nunca, por eso esos casos son más estables. Otro factor posible es la arquitectura híbrida del procesador, el sistema puede ubicar los hilos en núcleos de rendimiento o de eficiencia, que tienen velocidades distintas, y esa ubicación puede cambiar entre ejecuciones.

Limitaciones que deben tenerse en cuenta al interpretar los resultados:

-El tiempo medido incluye el trabajo del agente (leer el CSV y escribir en el pipe), no solo el del servidor.

-Un agente termina cuando su último mensaje entra al pipe; en ese momento pueden quedar hasta unos 910 mensajes (2.5 % de la carga) sin procesar, por lo que el tiempo real del servidor es ligeramente mayor que el medido.

-Las pruebas se ejecutaron en WSL 2 sobre un Intel Core Ultra 7 265KF de 20 núcleos, compartiendo el procesador con Windows. Los valores absolutos dependen de la máquina; las comparaciones entre configuraciones son confiables porque todas se midieron en las mismas condiciones y con 30 repeticiones.

7. Conclusiones

-Los tres componentes cumplen el enunciado. Los 33 casos de prueba funcionales dieron el resultado esperado, dando asi que el generador respeta la Tabla 1, el agente valida su archivo y reporta el progreso, y el servidor valida sus argumentos, maneja el pipe nominal y el límite de agentes como se especificó.

-El cálculo del ISE es correcto, incluso en los casos límite. El resultado coincidió con el cálculo manual en todos los casos, incluido el 50 % exacto, que da NEUTRO gracias a comparar el valor redondeado, y los sectores sin encuestas, cuyo peso se reparte entre los demás.

-El tamaño del buffer determina el rendimiento. Un buffer de 100 posiciones procesa unas 25 veces más rápido que uno de 1, porque reduce los bloqueos y cambios de contexto entre el hilo recolector y los hilos de los agentes. La mejora es decreciente, así que basta un buffer del orden de 100, con un costo de memoria de unos 7 KB por agente.

-Agregar agentes no aumenta la capacidad del servidor. El tiempo crece en proporción a la cantidad de agentes y la velocidad se mantiene constante, porque un solo pipe y un solo hilo recolector procesan los mensajes de forma secuencial. Para escalar habría que reducir esa parte secuencial, no agregar más hilos.

-El buffer compensa la concurrencia. Con un buffer adecuado, el servidor atiende el máximo de 4 agentes del enunciado más rápido de lo que atiende a un solo agente con un buffer mínimo.